Sveučilište Jurja Dobrile u Puli
Fakultet informatike u Puli

EtherItems

Dokumentacija

Pula, lipanj 2026. godine

Sveučilište Jurja Dobrile u Puli
Fakultet informatike u Puli

Luka Benković

EtherItems

Dokumentacija

JMBAG: 0152208610, online student
Studijski smjer: Sveučilišni diplomski studij Informatika
Kolegij: Blockchain aplikacije
Znanstveno područje: Društvene znanosti
Znanstveno polje: Informacijske i komunikacijske znanosti
Znanstvena grana: Informacijski sustavi i informatologija
Mentor: doc. dr. sc. Nikola Tanković

Sadržaj

1. Uvod

Na Blockchain-u postoji velik broj web3 igrica, koje korisnici lako mogu igrati tako da povežu svoje novčanike sa odgovarajućom valutom i vezom na odgovarajući Blockchain. Međutim, mnoge od tih igrica rade u svom okruženju gdje stvari (NFT) stečeni u igrici imaju vrijednost samo u toj igrici. Cilj ovog projekta je za razliku od gore navedenog, stvoriti verziju pametnog ugovora koji umjesto za jednu igricu, "minta" stvari koji su direktno povezani na blockchain sa pripadajućim detaljima (ATK, RARITY, DEFENSE, MAGIC, DURABILITY...) kako bi bilo moguće koristiti ih u bilo kojoj drugoj web3 igrici.

2. Korištene tehnologije

U razvoju projekta korišten je niz različitih tehnologija, za samu implementaciju pametnih ugovora, kao i klijentskog dijela za vizualni prikaz "mintanja" stvari, mogućnosti, i za potrebe demonstracije korištenja stvari u igrici kratka reprezentacija igrice.

1. Pametni ugovor:

    - Solidity 0.8.28

    - OpenZeppelin Contracts v5

2. Okruženje za razvoj na blockchainu

    - Hardhat 3

    - Hardhat Ignition

    - hardhat-toolbox-mocha-ethers

    - hardhat-ignition-ethers

3. Testing

    - Mocha

    - Chai

    - ethers.js v6

4. Klijentski dio

    - React 18

    - TypeScript

    - Vite 7

    - ethers.js v6

5. Novčanik / Interakcija sa blockchainom

    - MetaMask

    - ERC-721 standard

3. Struktura projekta

Struktura projekta je podijeljena u dva dijela. Dio za pametne ugovore i klijentski dio (frontend). Zbog jednostavnosti prezentacije smješteni su u isti direktorij ali rade odvojeno. Klijentski dio je smješten u poddirektorij /frontend i normalno se pokreće kao web aplikacija, dok je dio za pametne ugovore smješten u root direktorij / . Međusobno komuniciraju koristeći ethers.js.

3.1. Pametni ugovori

Pametni ugovori nalaze se u direktoriju /contracts, točnije, nalaze se dva pametna ugovora. ItemRegistry.sol i DungeonBattle.sol, te služe u različite svrhe. ItemRegistry.sol sadrži sve detalje i funkcije vezane za generiranje "mintanje" stvari, interakciju sa stvarima (smanjenje izdržljivosti (DURABILITY) stvari nakon bitke...), popravljanje stvari, dohvaćanje korisnikovih stvari i njihovih podataka, dok DungeonBattle.sol sadrži podatke i funkcije vezane za prezentaciju korištenja stvari u bitkama, interakciju između njih i sl.s

3.1.1. ItemRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract ItemRegistry is ERC721, Ownable {
    uint256 public constant MINT_PRICE = 0.01 ether;

    enum ItemType { SWORD, SHIELD, STAFF, BOOTS }
    enum Rarity { COMMON, UNCOMMON, RARE, EPIC, LEGENDARY }

    struct Item {
        ItemType itemType;
        Rarity rarity;
        uint8 attack;
        uint8 defense;
        uint8 magic;
        uint8 speed;
        uint8 durability;
        uint256 mintedAt;
    }

    mapping(uint256 => Item) public items;
    uint256 private _nextTokenId;
    address public dungeonContract;
```

```
    event ItemMinted(address indexed owner, uint256 indexed tokenId, ItemType
itemType, Rarity rarity);
    event DurabilityReduced(uint256 indexed tokenId, uint8 newDurability);
    event ItemRepaired(uint256 indexed tokenId, uint8 newDurability);

    constructor() ERC721("RPG Items", "RPGI") Ownable(msg.sender) {}

    modifier onlyDungeon() {
        require(msg.sender == dungeonContract, "ItemRegistry: caller is not the
dungeon contract");
        _;
    }

    function setDungeonContract(address _dungeon) external onlyOwner {
        require(_dungeon != address(0), "ItemRegistry: zero address");
        dungeonContract = _dungeon;
    }

    function mintItem(ItemType _itemType) external payable returns (uint256) {
        require(msg.value >= MINT_PRICE, "ItemRegistry: insufficient ETH");

        uint256 tokenId = _nextTokenId++;

        bytes32 seed = keccak256(abi.encodePacked(
            block.prevrandao,
            block.timestamp,
            msg.sender,
            tokenId
        ));

        Rarity rarity = _rollRarity(seed);
        Item memory newItem = _generateStats(_itemType, rarity, seed);

        items[tokenId] = newItem;
        _safeMint(msg.sender, tokenId);

        emit ItemMinted(msg.sender, tokenId, _itemType, rarity);
        return tokenId;
    }

    function _rollRarity(bytes32 seed) internal pure returns (Rarity) {
        uint8 roll = uint8(seed[0]) % 100;
        if (roll < 50) return Rarity.COMMON;
        if (roll < 75) return Rarity.UNCOMMON;
```

```
        if (roll < 90) return Rarity.RARE;
        if (roll < 98) return Rarity.EPIC;
        return Rarity.LEGENDARY;
    }

    function _rarityMultiplier(Rarity _rarity) internal pure returns (uint8) {
        if (_rarity == Rarity.COMMON)   return 10;
        if (_rarity == Rarity.UNCOMMON) return 13;
        if (_rarity == Rarity.RARE)     return 16;
        if (_rarity == Rarity.EPIC)     return 20;
        return 25;
    }

    function _generateStats(ItemType _type, Rarity _rarity, bytes32 seed) internal view
returns (Item memory) {
        uint8 mult = _rarityMultiplier(_rarity);
        uint8 statA = _scale(uint8(seed[1]) % 20 + 1, mult);
        uint8 statB = _scale(uint8(seed[2]) % 20 + 1, mult);
        uint8 durability = uint8(seed[3]) % 31 + 70;
        uint256 ts = block.timestamp;

        if (_type == ItemType.SWORD)  return Item(_type, _rarity, statA, 0, 0,
durability, ts);
        if (_type == ItemType.SHIELD) return Item(_type, _rarity, 0, statA, 0, 0, durability,
ts);
        if (_type == ItemType.STAFF)  return Item(_type, _rarity, statA, 0, statB, 0,
durability, ts);
        return Item(_type, _rarity, 0, 0, 0, statA, durability, ts); // BOOTS
    }

    function _scale(uint8 base, uint8 mult) internal pure returns (uint8) {
        uint16 result = uint16(base) * uint16(mult) / 10;
        return result > 100 ? 100 : uint8(result);
    }

    function reduceDurability(uint256 tokenId) external onlyDungeon {
        Item storage item = items[tokenId];
        require(item.durability > 0, "ItemRegistry: item is already broken");
        uint8 reduction = item.durability >= 5 ? 5 : item.durability;
        item.durability -= reduction;
        emit DurabilityReduced(tokenId, item.durability);
    }

    function repairItem(uint256 tokenId) external payable {
```

```
        require(ownerOf(tokenId) == msg.sender, "ItemRegistry: not your item");
        require(msg.value >= 0.005 ether, "ItemRegistry: insufficient repair fee");
        require(items[tokenId].durability < 100, "ItemRegistry: already at full
durability");
        items[tokenId].durability = 100;
        emit ItemRepaired(tokenId, 100);
    }

    function getItem(uint256 tokenId) external view returns (Item memory) {
        return items[tokenId];
    }

    function getPowerScore(uint256 tokenId) external view returns (uint256) {
        Item memory item = items[tokenId];
        if (item.durability == 0) return 0;
        uint256 statTotal = item.attack + item.defense + item.magic + item.speed;
        return (statTotal * item.durability) / 100;
    }

    function withdraw() external onlyOwner {
        uint256 balance = address(this).balance;
        require(balance > 0, "ItemRegistry: nothing to withdraw");
        payable(owner()).transfer(balance);
    }
}
```

ItemRegistry.sol nam je glavna datoteka i podloga za ovaj projekt. Ona sadrži sve detalje vezane za naše stvari, definiciju strukture podataka, sve funkcije potrebne za rad sa stvarima i postavljanje podataka na blockchain. Glavni podatak koji će se spremati na blockchain je Item sa strukturom koju smo definirali sa struct Item koji sadrži podatke o stvarima. Podatke kao što su attack, defense, magic, speed, durability, rarity, itemType i mintedAt odnosno vrijeme kada je stvar "mintana" u obliku Unix timestamp. Sama ova struktura i način spremanja podataka je temelj cijelog projekta, odnosno umjesto da su podaci spremljeni u bazi podataka firme koja je napravila igricu i stvari, podaci su spremljeni direktno na blockchainu. Funkcija mintItem nam je glavna funkcija u ugovoru koja je odgovorna na "mintanje" stvari. Kako bi osigurali "pseudonasumičnost" generiramo seed iz kombinacije više izvora točnije iz podataka blocka, adrese pozivatelja, i tokenId-a. Pomoću generiranog seed-a generiramo podatke o stvari. Prije nego krenemo u generiranje samih detalja stvari prvo moramo odrediti "rarity" odnosno koliko je rijetka stvar koju generiramo jer pomoću tog podataka naknadno generiramo ostale podatke kao što su attack, magic i slično ovisno o tipu stvari koja je generirana, te na temelju rijetkosti stvari skaliramo podatke koje stvar prima. Nakon što smo to odradili i dobili našu generirano stvar spremamo ju u items što nam

je baza podataka na blockchainu. Koristimo funkciju _safeMint iz OpenZepellin-a koja generira NFT i pridružuje ga vlasniku. Osim mintItem funkcije također imamo pomoćne funkcije koje smo koristili u glavnoj funkciji za generiranje podataka o stvarima. Osim toga imamo i funkcije za promjenu podataka stvari. ReduceDurability funkcija smanjuje durability odnosno izdržljivost stvari za 5 nakon svake bitke do minimuma od 0. RepairItem funkcija popravlja stvar odnosno podiže izdržljivost stvari na 100 tj maksimum po cijeni od 0.005 ETH za potpuni popravak. Ukoliko je stvar već na maksimalnoj izdržljivosti vraća poruku greške. Obje funkcije emitaju event koji kasnije dohvaćamo u klijentskom dijelu aplikacije. GetItem i GetPowerScore funkcije vraćaju nam podatke o stvarima. Withdraw funkcija služi vlasniku aplikacije za prikupljanje svih sredstava koje je aplikacija generirala. Važno je napomenuti u ovom dijelu aplikacije modifier onlyDungeon. On nam osigurava da se samo kroz bitku može smanjiti izdržljivost stvari, u suprotnom svaki korisnik blockchaina može pozivom reduceDurability smanjiti izdržljivost stvari na 0. Taj dio ugovora bi se kroz nadogradnju mogao proširiti sa nekakvom "bijelom listom" aplikacija/igrica koje imaju ovlaštenje mijenjati izdržljivost stvari. Čak i sa trenutačnim ograničenjem glavna ideja projekta ostaje ista. Svaka aplikacija/igrica ima pristup podacima o stvarima korisnika te ih može koristiti, ili stvarati nove.

3.1.2. DungeonBattle.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "@openzeppelin/contracts/access/Ownable.sol";

interface IItemRegistry {
    enum ItemType { SWORD, SHIELD, STAFF, BOOTS }
    enum Rarity { COMMON, UNCOMMON, RARE, EPIC, LEGENDARY }

    struct Item {
        ItemType itemType;
        Rarity rarity;
        uint8 attack;
        uint8 defense;
        uint8 magic;
        uint8 speed;
        uint8 durability;
        uint256 mintedAt;
    }

    function ownerOf(uint256 tokenId) external view returns (address);
    function getItem(uint256 tokenId) external view returns (Item memory);
    function getPowerScore(uint256 tokenId) external view returns (uint256);
    function reduceDurability(uint256 tokenId) external;
}

contract DungeonBattle is Ownable {
    uint256 public constant ENTRY_FEE = 0.005 ether;

    enum Difficulty { EASY, MEDIUM, HARD, LEGENDARY }

    struct Monster {
        string name;
        uint256 power;
        uint256 rewardEth;
    }

    struct BattleRecord {
        address player;
        uint256 itemTokenId;
        uint256 playerPower;
        uint256 monsterPower;
```

```
        bool playerWon;
        uint256 timestamp;
        string monsterName;
    }

    IItemRegistry public itemRegistry;

    BattleRecord[] public battleHistory;

    mapping(address => uint256) public wins;
    mapping(address => uint256) public losses;

    mapping(address => uint256) public pendingRewards;

    event BattleStarted(address indexed player, uint256 indexed tokenId, string
monsterName);
    event BattleResolved(address indexed player, bool playerWon, uint256
playerPower, uint256 monsterPower);
    event RewardClaimed(address indexed player, uint256 amount);

    constructor(address _itemRegistry) Ownable(msg.sender) {
        require(_itemRegistry != address(0), "DungeonBattle: zero address");
        itemRegistry = IItemRegistry(_itemRegistry);
    }

    function enterDungeon(uint256 tokenId, Difficulty difficulty) external payable {
        require(msg.value >= ENTRY_FEE, "DungeonBattle: insufficient entry fee");
        require(
            itemRegistry.ownerOf(tokenId) == msg.sender,
            "DungeonBattle: you don't own this item"
        );

        uint256 playerPower = itemRegistry.getPowerScore(tokenId);
        require(playerPower > 0, "DungeonBattle: item is broken, repair it first");

        Monster memory monster = _spawnMonster(difficulty, tokenId);

        emit BattleStarted(msg.sender, tokenId, monster.name);

        uint256 variance = _rollVariance(tokenId, msg.sender);
        uint256 effectivePower = (playerPower * variance) / 100;

        bool playerWon = effectivePower >= monster.power;
```

```
        battleHistory.push(BattleRecord({
            player: msg.sender,
            itemTokenId: tokenId,
            playerPower: effectivePower,
            monsterPower: monster.power,
            playerWon: playerWon,
            timestamp: block.timestamp,
            monsterName: monster.name
        }));

        if (playerWon) {
            wins[msg.sender]++;
            pendingRewards[msg.sender] += monster.rewardEth;
        } else {
            losses[msg.sender]++;
        }

        itemRegistry.reduceDurability(tokenId);

        emit BattleResolved(msg.sender, playerWon, effectivePower, monster.power);
    }

    function _spawnMonster(
        Difficulty difficulty,
        uint256 tokenId
    ) internal view returns (Monster memory) {
        uint8 idx = uint8(
            uint256(keccak256(abi.encodePacked(block.timestamp, tokenId))) % 4
        );

        if (difficulty == Difficulty.EASY) {
            string[4] memory names = ["Slime", "Goblin", "Rat", "Mushroom"];
            return Monster(names[idx], 15, 0.003 ether);
        } else if (difficulty == Difficulty.MEDIUM) {
            string[4] memory names = ["Orc", "Skeleton", "Troll", "Bandit"];
            return Monster(names[idx], 30, 0.006 ether);
        } else if (difficulty == Difficulty.HARD) {
            string[4] memory names = ["Dark Knight", "Wyvern", "Necromancer",
"Golem"];
            return Monster(names[idx], 55, 0.012 ether);
        } else {
            string[4] memory names = ["Dragon", "Lich King", "Demon Lord", "Ancient
God"];
            return Monster(names[idx], 90, 0.025 ether);
```

```
        }
    }

    function _rollVariance(uint256 tokenId, address player) internal view returns
(uint256) {
        uint256 roll = uint256(keccak256(abi.encodePacked(
            block.prevrandao,
            block.timestamp,
            tokenId,
            player
        ))) % 41;

        return 80 + roll;
    }

    function claimRewards() external {
        uint256 amount = pendingRewards[msg.sender];
        require(amount > 0, "DungeonBattle: no rewards to claim");

        pendingRewards[msg.sender] = 0;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "DungeonBattle: transfer failed");

        emit RewardClaimed(msg.sender, amount);
    }

    function getPlayerStats(address player)
        external
        view
        returns (uint256 playerWins, uint256 playerLosses)
    {
        return (wins[player], losses[player]);
    }

    function getTotalBattles() external view returns (uint256) {
        return battleHistory.length;
    }

    function getRecentBattles(uint256 count)
        external
        view
        returns (BattleRecord[] memory)
    {
```

```
        uint256 total = battleHistory.length;
        uint256 resultCount = count > total ? total : count;

        BattleRecord[] memory result = new BattleRecord[](resultCount);
        for (uint256 i = 0; i < resultCount; i++) {
            result[i] = battleHistory[total - resultCount + i];
        }
        return result;
    }

    function fundContract() external payable onlyOwner {}

    function withdraw() external onlyOwner {
        uint256 balance = address(this).balance;
        require(balance > 0, "DungeonBattle: nothing to withdraw");
        payable(owner()).transfer(balance);
    }
}
```

Ovaj pametni ugovor za bitke će nam poslužiti za demonstraciju korištenja stvari koje smo generirali na blockchainu. Ova igrica će biti jednostavnog tipa. Imat ćemo mogućnost "mintanja" stvari, pregled svih naših stvari koje imamo, i mogućnost ulaska u bitku sa njima uz mogućnost osvajanja ETH kroz bitke, pregled bitaka koje smo odradili i mogućnost preuzimanja osvojenog ETH-a. Prije nego što krenemo u izradu funkcija igrice, definiramo interface za ItemRegistry jer moramo koristiti funkcije koje su tamo definirane. Umjesto da importamo cijeli ItemRegistry ugovor definiramo samo funkcije koje su nam potrebne iz njega, kako bi Solidity znao kako da točno enkodira pozive tih funkcija. IitemRegistry varijabla sadrži adresu gdje je deploy-an ItemRegistry ugovor te je definirana tipa kao interface koji smo definirali kako bi mogli direktno pozvati funkcije koje sadrži. Glavna funkcija u ovom ugovoru nam je enterDungeon odnosno ulazak u bitku sa stvarima. Prije nego dopustimo ulazak u bitku moramo osigurati da igrač koji ulazi u bitku ima dovoljnu količinu ETH-a za platiti ulazak u bitku, da stvar nije totalno degradirana odnosno da ima izdržljivost veću od 0 i da je igrač stvarni vlasnik te stvari. Nakon što smo to osigurali imamo pomoćnu funkciju "stvaranja" čudovišta protiv kojeg se igrač bori sa svojom stvari. Jačina tog čudovišta ovisi o težini bitku u koju je igrač odlučio ući. Kako bi nastavili sa temom "slučajnosti" i simulacijom igrice uvest ćemo funkciju varijance odnosno _rollVariance. Tom funkcijom ćemo dobiti efektivnu jačinu igračeve stvari koja varira od 80-120% jačine. Na taj način ćemo dobiti slučajnost i eliminirati determinizam u bitkama, gdje ako je jačina stvari npr. 18 a najslabije čudoviše koje je moguće stvoriti u bitci je 15, još uvijek postoji mogućnost da igrač izgubi bitku ako je igrom slučaja dobio varijancu od 0 i tako mu je efektivna snaga stvari zapravo

samo 80% od snage koju stvar ima. Na taj način smo uveli "RNG" u igricu kako bi bila zanimljivija. Nadalje, imamo funkcije za pregled bitki koje je igrač odigrao, da li je pobjedio ili izgubio, opciju preuzimanja osvojenih ETH-a kroz bitke, i funkcije vezane za vlasnika ugovora za preuzimanje ETH-a koje je igrica zaradila. U funkciji claimRewards, važno je napomenuti redoslijed izvođenja operacija. Kako bi osigurali da ne može doći do ponavljanja više puta preuzimanja zarađenih ETH-a, prije samo transfera sredstava igraču, postavljamo zarađene nagrade na 0, i tek onda radimo transfer.

3.2. Testovi

3.2.1. rpg.test.ts

```
import { expect } from "chai";
import hre from "hardhat";

const ItemType = { SWORD: 0, SHIELD: 1, STAFF: 2, BOOTS: 3 };
const Difficulty = { EASY: 0, MEDIUM: 1, HARD: 2, LEGENDARY: 3 };

async function deploy() {
  const { ethers } = await hre.network.create();

  const MINT_PRICE = ethers.parseEther("0.01");
  const ENTRY_FEE = ethers.parseEther("0.005");
  const FUND_AMOUNT = ethers.parseEther("1.0");

  const [owner, player1, player2] = await ethers.getSigners();

  const ItemRegistryFactory = await ethers.getContractFactory("ItemRegistry");
  const itemRegistry = await ItemRegistryFactory.deploy();
  await itemRegistry.waitForDeployment();

  const DungeonBattleFactory = await ethers.getContractFactory(
    "DungeonBattle"
  );
  const dungeonBattle = await DungeonBattleFactory.deploy(
    await itemRegistry.getAddress()
  );
  await dungeonBattle.waitForDeployment();

  await itemRegistry.setDungeonContract(await dungeonBattle.getAddress());
  await dungeonBattle.fundContract({ value: FUND_AMOUNT });
```

```
  return {
    ethers,
    MINT_PRICE,
    ENTRY_FEE,
    itemRegistry,
    dungeonBattle,
    owner,
    player1,
    player2
  };
}

describe("ItemRegistry", function () {
  describe("Deployment", function () {
    it("sets the deployer as owner", async function () {
      const { itemRegistry, owner } = await deploy();
      expect(await itemRegistry.owner()).to.equal(owner.address);
    });

    it("has correct ERC721 name and symbol", async function () {
      const { itemRegistry } = await deploy();
      expect(await itemRegistry.name()).to.equal("RPG Items");
      expect(await itemRegistry.symbol()).to.equal("RPGI");
    });

    it("links to DungeonBattle correctly", async function () {
      const { itemRegistry, dungeonBattle } = await deploy();
      expect(await itemRegistry.dungeonContract()).to.equal(
        await dungeonBattle.getAddress()
      );
    });
  });

  describe("Minting", function () {
    it("mints a sword and assigns it to the caller", async function () {
      const { itemRegistry, player1, MINT_PRICE } = await deploy();
      await expect(
        itemRegistry
          .connect(player1)
          .mintItem(ItemType.SWORD, { value: MINT_PRICE })
      ).to.emit(itemRegistry, "ItemMinted");
      expect(await itemRegistry.ownerOf(0)).to.equal(player1.address);
    });
```

```
it("reverts when ETH sent is below MINT_PRICE", async function () {
  const { ethers, itemRegistry, player1 } = await deploy();
  await expect(
    itemRegistry.connect(player1).mintItem(ItemType.SWORD, {
      value: ethers.parseEther("0.001")
    })
  ).to.be.revertedWith("ItemRegistry: insufficient ETH");
});

it("mints all four item types without reverting", async function () {
  const { ethers, itemRegistry, player1, MINT_PRICE } =
    await deploy();
  for (const typeId of Object.values(ItemType)) {
    await expect(
      itemRegistry
        .connect(player1)
        .mintItem(typeId, { value: MINT_PRICE })
    ).to.not.be.revert(ethers);
  }
});

it("assigns non-zero primary stat per item type", async function () {
  const { itemRegistry, player1, MINT_PRICE } = await deploy();
  await itemRegistry
    .connect(player1)
    .mintItem(ItemType.SWORD, { value: MINT_PRICE });
  await itemRegistry
    .connect(player1)
    .mintItem(ItemType.SHIELD, { value: MINT_PRICE });
  await itemRegistry
    .connect(player1)
    .mintItem(ItemType.STAFF, { value: MINT_PRICE });
  await itemRegistry
    .connect(player1)
    .mintItem(ItemType.BOOTS, { value: MINT_PRICE });

  expect((await itemRegistry.getItem(0)).attack).to.be.gt(0);
  expect((await itemRegistry.getItem(1)).defense).to.be.gt(0);
  expect((await itemRegistry.getItem(2)).magic).to.be.gt(0);
  expect((await itemRegistry.getItem(3)).speed).to.be.gt(0);
});

it("starts every item with durability 70–100", async function () {
  const { itemRegistry, player1, MINT_PRICE } = await deploy();
```

```
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      const item = await itemRegistry.getItem(0);
      expect(item.durability).to.be.gte(70);
      expect(item.durability).to.be.lte(100);
    });

    it("increments tokenId with each mint", async function () {
      const { itemRegistry, player1, MINT_PRICE } = await deploy();
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SHIELD, { value: MINT_PRICE });
      expect(await itemRegistry.ownerOf(0)).to.equal(player1.address);
      expect(await itemRegistry.ownerOf(1)).to.equal(player1.address);
    });
  });

  describe("Power Score", function () {
    it("returns a positive power score for a healthy item", async function () {
      const { itemRegistry, player1, MINT_PRICE } = await deploy();
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      expect(await itemRegistry.getPowerScore(0)).to.be.gt(0);
    });
  });

  describe("Repair", function () {
    it("restores durability to 100 after battle", async function () {
      const {
        ethers,
        itemRegistry,
        dungeonBattle,
        player1,
        MINT_PRICE,
        ENTRY_FEE
      } = await deploy();
      const REPAIR_FEE = ethers.parseEther("0.005");
      await itemRegistry
        .connect(player1)
```

```
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      await dungeonBattle
        .connect(player1)
        .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
      await itemRegistry
        .connect(player1)
        .repairItem(0, { value: REPAIR_FEE });
      expect((await itemRegistry.getItem(0)).durability).to.equal(100);
    });

    it("reverts if called by non-owner of the item", async function () {
      const { ethers, itemRegistry, player1, player2, MINT_PRICE } =
        await deploy();
      const REPAIR_FEE = ethers.parseEther("0.005");
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      await expect(
        itemRegistry
          .connect(player2)
          .repairItem(0, { value: REPAIR_FEE })
      ).to.be.revertedWith("ItemRegistry: not your item");
    });
  });

  describe("Access Control", function () {
    it("reverts when non-dungeon calls reduceDurability", async function () {
      const { itemRegistry, player1, MINT_PRICE } = await deploy();
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      await expect(
        itemRegistry.connect(player1).reduceDurability(0)
      ).to.be.revertedWith(
        "ItemRegistry: caller is not the dungeon contract"
      );
    });

    it("reverts when non-owner calls setDungeonContract", async function () {
      const { ethers, itemRegistry, dungeonBattle, player1 } =
        await deploy();
      await expect(
        itemRegistry
          .connect(player1)
```

```
            .setDungeonContract(await dungeonBattle.getAddress())
        ).to.be.revert(ethers);
      });
    });

    describe("Withdrawal", function () {
      it("lets owner withdraw mint fees", async function () {
        const { ethers, itemRegistry, owner, player1, MINT_PRICE } =
          await deploy();
        await itemRegistry
          .connect(player1)
          .mintItem(ItemType.SWORD, { value: MINT_PRICE });
        const before = await ethers.provider.getBalance(owner.address);
        await itemRegistry.connect(owner).withdraw();
        const after = await ethers.provider.getBalance(owner.address);
        expect(after).to.be.gt(before);
      });
    });
});

describe("DungeonBattle", function () {
    describe("Deployment", function () {
      it("points to the correct ItemRegistry", async function () {
        const { itemRegistry, dungeonBattle } = await deploy();
        expect(await dungeonBattle.itemRegistry()).to.equal(
          await itemRegistry.getAddress()
        );
      });
    });

    describe("Enter Dungeon", function () {
      it("emits BattleStarted on entry", async function () {
        const {
          itemRegistry,
          dungeonBattle,
          player1,
          MINT_PRICE,
          ENTRY_FEE
        } = await deploy();
        await itemRegistry
          .connect(player1)
          .mintItem(ItemType.SWORD, { value: MINT_PRICE });
        await expect(
          dungeonBattle
```

```
            .connect(player1)
            .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE })
    ).to.emit(dungeonBattle, "BattleStarted");
});

it("reverts if caller doesn't own the item", async function () {
    const {
        itemRegistry,
        dungeonBattle,
        player1,
        player2,
        MINT_PRICE,
        ENTRY_FEE
    } = await deploy();
    await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
    await expect(
        dungeonBattle
            .connect(player2)
            .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE })
    ).to.be.revertedWith("DungeonBattle: you don't own this item");
});

it("reverts if entry fee is not paid", async function () {
    const { ethers, itemRegistry, dungeonBattle, player1, MINT_PRICE } =
        await deploy();
    await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
    await expect(
        dungeonBattle
            .connect(player1)
            .enterDungeon(0, Difficulty.EASY, {
                value: ethers.parseEther("0.001")
            })
    ).to.be.revertedWith("DungeonBattle: insufficient entry fee");
});

it("records the battle in history", async function () {
    const {
        itemRegistry,
        dungeonBattle,
        player1,
```

```
      MINT_PRICE,
      ENTRY_FEE
    } = await deploy();
    await itemRegistry
      .connect(player1)
      .mintItem(ItemType.SWORD, { value: MINT_PRICE });
    await dungeonBattle
      .connect(player1)
      .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
    expect(await dungeonBattle.getTotalBattles()).to.equal(1);
  });

  it("reduces item durability by 5", async function () {
    const {
      itemRegistry,
      dungeonBattle,
      player1,
      MINT_PRICE,
      ENTRY_FEE
    } = await deploy();
    await itemRegistry
      .connect(player1)
      .mintItem(ItemType.SWORD, { value: MINT_PRICE });
    const before = Number((await itemRegistry.getItem(0)).durability);
    await dungeonBattle
      .connect(player1)
      .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
    const after = Number((await itemRegistry.getItem(0)).durability);
    expect(after).to.equal(before - 5);
  });

  it("wins + losses sum to 1 after one battle", async function () {
    const {
      itemRegistry,
      dungeonBattle,
      player1,
      MINT_PRICE,
      ENTRY_FEE
    } = await deploy();
    await itemRegistry
      .connect(player1)
      .mintItem(ItemType.SWORD, { value: MINT_PRICE });
    await dungeonBattle
      .connect(player1)
```

```
        .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
      const [wins, losses] = await dungeonBattle.getPlayerStats(
        player1.address
      );
      expect(wins + losses).to.equal(1n);
    });

    it("accepts all four difficulty levels", async function () {
      const {
        ethers,
        itemRegistry,
        dungeonBattle,
        player1,
        MINT_PRICE,
        ENTRY_FEE
      } = await deploy();
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      for (const diff of Object.values(Difficulty)) {
        await expect(
          dungeonBattle
            .connect(player1)
            .enterDungeon(0, diff, { value: ENTRY_FEE })
        ).to.not.be.revert(ethers);
      }
    });
});

describe("Rewards", function () {
    it("reverts claimRewards when nothing to claim", async function () {
      const { dungeonBattle, player1 } = await deploy();
      await expect(
        dungeonBattle.connect(player1).claimRewards()
      ).to.be.revertedWith("DungeonBattle: no rewards to claim");
    });

    it("pays ETH to player after winning", async function () {
      const {
        ethers,
        itemRegistry,
        dungeonBattle,
        player1,
        MINT_PRICE,
```

```
      ENTRY_FEE
    } = await deploy();
    const REPAIR_FEE = ethers.parseEther("0.005");
    await itemRegistry
      .connect(player1)
      .mintItem(ItemType.SWORD, { value: MINT_PRICE });

    let won = false;
    for (let i = 0; i < 20 && !won; i++) {
      const item = await itemRegistry.getItem(0);
      if (Number(item.durability) <= 10) {
        await itemRegistry
          .connect(player1)
          .repairItem(0, { value: REPAIR_FEE });
      }
      await dungeonBattle
        .connect(player1)
        .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
      if ((await dungeonBattle.pendingRewards(player1.address)) > 0n)
        won = true;
    }

    if (won) {
      const before = await ethers.provider.getBalance(
        player1.address
      );
      await dungeonBattle.connect(player1).claimRewards();
      const after = await ethers.provider.getBalance(player1.address);
      expect(after).to.be.gt(before);
    }
  });
});

describe("Battle History", function () {
  it("getRecentBattles returns the correct count", async function () {
    const {
      itemRegistry,
      dungeonBattle,
      player1,
      MINT_PRICE,
      ENTRY_FEE
    } = await deploy();
    await itemRegistry
      .connect(player1)
```

```
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      for (let i = 0; i < 3; i++) {
        await dungeonBattle
          .connect(player1)
          .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
      }
      expect((await dungeonBattle.getRecentBattles(2)).length).to.equal(
        2
      );
    });

    it("stores the player address in battle history", async function () {
      const {
        itemRegistry,
        dungeonBattle,
        player1,
        MINT_PRICE,
        ENTRY_FEE
      } = await deploy();
      await itemRegistry
        .connect(player1)
        .mintItem(ItemType.SWORD, { value: MINT_PRICE });
      await dungeonBattle
        .connect(player1)
        .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
      const battles = await dungeonBattle.getRecentBattles(1);
      expect(battles[0].player).to.equal(player1.address);
    });
});

describe("Full Game Loop", function () {
    it("completes mint -> battle -> repair -> claim", async function () {
      const {
        ethers,
        itemRegistry,
        dungeonBattle,
        player1,
        MINT_PRICE,
        ENTRY_FEE
      } = await deploy();
      const REPAIR_FEE = ethers.parseEther("0.005");

      await itemRegistry
        .connect(player1)
```

```
            .mintItem(ItemType.SWORD, { value: MINT_PRICE });
        expect(await itemRegistry.ownerOf(0)).to.equal(player1.address);
        const item = await itemRegistry.getItem(0);
        console.log(
            `\nMinted Sword — Attack: ${item.attack}, Durability: ${item.durability}`
        );

        await dungeonBattle
            .connect(player1)
            .enterDungeon(0, Difficulty.EASY, { value: ENTRY_FEE });
        const [wins, losses] = await dungeonBattle.getPlayerStats(
            player1.address
        );
        console.log(`Battle — Wins: ${wins}, Losses: ${losses}`);

        const afterBattle = await itemRegistry.getItem(0);
        expect(Number(afterBattle.durability)).to.be.lt(
            Number(item.durability)
        );

        await itemRegistry
            .connect(player1)
            .repairItem(0, { value: REPAIR_FEE });
        expect((await itemRegistry.getItem(0)).durability).to.equal(100);
        console.log(`Repaired to 100`);

        const pending = await dungeonBattle.pendingRewards(player1.address);
        if (pending > 0n) {
            await dungeonBattle.connect(player1).claimRewards();
            console.log(`Claimed ${ethers.formatEther(pending)} ETH`);
        }
        console.log(`Full loop complete!\n`);
    });
  });
});
```

U ovoj datoteci definiramo testove koje provodimo na ugovorima prije samog deploy-a na blockchain kako bi osigurali da sve funkcije ugovora rade kako je predviđeno, što radimo na način da tekstualno opišemo što očekujemo od te funkcije odnosno od toga testa da se treba dogoditi te pomoću funkcije expect iz "chai" biblioteke definiramo unutar testa što se treba dogoditi kada se funkcija izvede odnosno što funkcija treba vratiti u tom slučaju koji smo definirali. Glavna funkcija u ovoj datoteci je zapravo funkcija deploy koju pozivamo na početku svakog testa, kako bi svaki

test imao svoju instancu blockchaina te na taj način osiguravamo da ne dolazi do ometanja u izvođenju testova.

3.3. Deployment

3.3.1. Ignition

```
import { buildModule } from "@nomicfoundation/hardhat-ignition/modules";

const EtherItemsModule = buildModule("EtherItems", (m) => {
  const itemRegistry = m.contract("ItemRegistry");

  const dungeonBattle = m.contract("DungeonBattle", [itemRegistry]);

  m.call(itemRegistry, "setDungeonContract", [dungeonBattle], {
    id: "LinkContracts"
  });

  m.call(dungeonBattle, "fundContract", [], {
    id: "FundDungeon",
    value: 100_000_000_000_000_000n // 0.1 ETH u wei
  });

  return { itemRegistry, dungeonBattle };
});

export default EtherItemsModule;
```

Ova datoteka služi nam za deployment ugovora na blockchain. U ovom projektu ugovori će biti deploy-ani na lokalnu blockchain mrežu. Ovdje isto moramo paziti na redoslijed deploy-anja ugovora jer nam DungeonBattle ugovor ovisi o ItemRegistry ugovoru, moramo naravno prvo deploy-ati ItemRegistry ugovor te tek nakon toga možemo deployati DungeonBattle i navesti ItemRegistry kao dependancy, tj. Definiramo da DungeonBattle ugovor ovisi o tome da je ItemRegistry već deploy-an na blockchain. Nakon što su ugovori deploy-ani pozivamo dvije funkcije prvo pozivamo funkciju setDungeonContract koji nam definira našu igricu kao jedinu ovlaštenu za uređivanje podataka stvari odnosno promjenu izdržljivosti stvari kako smo definirali u ugovoru. Sljedeća funkcija je dodavanje sredstava sa vlasnikovog računa kako bi igrica mogla isplatiti nagrade igračima. Na kraju sami modul vraća adrese ugovora kako bi se mogle kasnije koristiti.

4. Klijentski dio

4.1. Utils

4.1.1. Contracts.ts

```
export const ADDRESSES: Record<
  number,
  { ItemRegistry: string; DungeonBattle: string }
> = {
  // Local Hardhat node
  31337: {
    ItemRegistry: "0x5FbDB2315678afecb367f032d93F642f64180aa3",
    DungeonBattle: "0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512"
  }
  // Sepolia testnet
  // 11155111: {
  //   ItemRegistry: "",
  //   DungeonBattle: "",
  // },
};

export const ITEM_REGISTRY_ABI = [
  "function mintItem(uint8 itemType) external payable returns (uint256)",
  "function getItem(uint256 tokenId) external view returns (tuple(uint8 itemType,
uint8 rarity, uint8 attack, uint8 defense, uint8 magic, uint8 speed, uint8 durability,
uint256 mintedAt))",
  "function getPowerScore(uint256 tokenId) external view returns (uint256)",
  "function repairItem(uint256 tokenId) external payable",
  "function ownerOf(uint256 tokenId) external view returns (address)",
  "function balanceOf(address owner) external view returns (uint256)",
  "function MINT_PRICE() external view returns (uint256)",
  "event ItemMinted(address indexed owner, uint256 indexed tokenId, uint8
itemType, uint8 rarity)",
  "event ItemRepaired(uint256 indexed tokenId, uint8 newDurability)"
] as const;

export const DUNGEON_BATTLE_ABI = [
  "function enterDungeon(uint256 tokenId, uint8 difficulty) external payable",
  "function claimRewards() external",
  "function getPlayerStats(address player) external view returns (uint256 wins,
uint256 losses)",
  "function getRecentBattles(uint256 count) external view returns (tuple(address
player, uint256 itemTokenId, uint256 playerPower, uint256 monsterPower, bool
playerWon, uint256 timestamp, string monsterName)[])",
```

```
  "function pendingRewards(address player) external view returns (uint256)",
  "function getTotalBattles() external view returns (uint256)",
  "function ENTRY_FEE() external view returns (uint256)",
  "event BattleStarted(address indexed player, uint256 indexed tokenId, string
monsterName)",
  "event BattleResolved(address indexed player, bool playerWon, uint256
playerPower, uint256 monsterPower)",
  "event RewardClaimed(address indexed player, uint256 amount)"
] as const;

export const ItemType = { SWORD: 0, SHIELD: 1, STAFF: 2, BOOTS: 3 } as const;
export const ItemTypeLabel = ["Sword", "Shield", "Staff", "Boots"] as const;
export const ItemTypeEmoji = ["⚔️", "🛡️", "🪄", "👢"] as const;

export const Rarity = {
  COMMON: 0,
  UNCOMMON: 1,
  RARE: 2,
  EPIC: 3,
  LEGENDARY: 4
} as const;
export const RarityLabel = [
  "Common",
  "Uncommon",
  "Rare",
  "Epic",
  "Legendary"
] as const;
export const RarityColor = [
  "#9ca3af",
  "#4ade80",
  "#60a5fa",
  "#c084fc",
  "#fbbf24"
] as const;

export const Difficulty = {
  EASY: 0,
  MEDIUM: 1,
  HARD: 2,
  LEGENDARY: 3
} as const;
export const DifficultyLabel = ["Easy", "Medium", "Hard", "Legendary"] as const;
export const DifficultyEmoji = ["🐣", "⚔️", "💀", "🐉"] as const;
```

```
export const DifficultyColor = [
    "#4ade80",
    "#facc15",
    "#f97316",
    "#ef4444"
] as const;

export interface ItemData {
    tokenId: number;
    itemType: number;
    rarity: number;
    attack: number;
    defense: number;
    magic: number;
    speed: number;
    durability: number;
    mintedAt: number;
    power: number;
}

export interface BattleRecord {
    player: string;
    itemTokenId: number;
    playerPower: number;
    monsterPower: number;
    playerWon: boolean;
    timestamp: number;
    monsterName: string;
}
```

Ova datoteka služi nam kao izvor jedinstvene istine i poveznica sa našim ugovorima na blockchain-u. U njoj definiramo poveznicu sa adresama na kojima se nalaze naši ugovori, ABI-je tj. Application Binary Interface gdje definiramo funkcije dostupne u tim ugovorima koje možemo pozvati sa frontenda. Također definiramo i tipove podataka koje ugovori očekuju kako ne bi došlo do neželjenih grešaka, što je i razlog zašto koristimo TypeScript kao jedan sloj sigurnosti da podatke koje šaljemo između frontenda i naših ugovora budu istog oblika/tipa.

4.2. Hooks

4.2.1. useWallet.ts

```
import { useState, useEffect, useCallback } from "react";
```

```ts
import { ethers } from "ethers";
import { ADDRESSES } from "../utils/contracts";

declare global {
  interface Window {
    ethereum?: {
      request: (args: {
        method: string;
        params?: unknown[];
      }) => Promise<unknown>;
      on: (event: string, handler: (...args: unknown[]) => void) => void;
      removeListener: (
        event: string,
        handler: (...args: unknown[]) => void
      ) => void;
    };
  }
}

export interface WalletState {
  provider: ethers.BrowserProvider | null;
  signer: ethers.JsonRpcSigner | null;
  address: string | null;
  chainId: number | null;
  balance: string | null;
  error: string | null;
  connecting: boolean;
  isSupported: boolean;
  connect: () => Promise<void>;
  refreshBalance: () => Promise<void>;
}

export function useWallet(): WalletState {
  const [provider, setProvider] = useState<ethers.BrowserProvider | null>(
    null
  );
  const [signer, setSigner] = useState<ethers.JsonRpcSigner | null>(null);
  const [address, setAddress] = useState<string | null>(null);
  const [chainId, setChainId] = useState<number | null>(null);
  const [balance, setBalance] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [connecting, setConnecting] = useState(false);

  const isSupported = Boolean(chainId && ADDRESSES[chainId]);
```

```
const refreshBalance = useCallback(async () => {
  if (!provider || !address) return;
  const bal = await provider.getBalance(address);
  setBalance(ethers.formatEther(bal));
}, [provider, address]);

const connect = useCallback(async () => {
  if (!window.ethereum) {
    setError("MetaMask not found. Install it from metamask.io");
    return;
  }
  setConnecting(true);
  setError(null);
  try {
    const prov = new ethers.BrowserProvider(window.ethereum);
    await prov.send("eth_requestAccounts", []);
    const sign = await prov.getSigner();
    const addr = await sign.getAddress();
    const network = await prov.getNetwork();
    const cId = Number(network.chainId);
    const bal = await prov.getBalance(addr);

    setProvider(prov);
    setSigner(sign);
    setAddress(addr);
    setChainId(cId);
    setBalance(ethers.formatEther(bal));
  } catch (e) {
    const msg = e instanceof Error ? e.message : "Connection failed";
    setError(msg);
  } finally {
    setConnecting(false);
  }
}, []);

useEffect(() => {
  if (!window.ethereum) return;

  const handleAccountsChanged = (...args: unknown[]) => {
    const accounts = args[0] as string[];
    if (accounts.length === 0) {
      setAddress(null);
      setSigner(null);
```

```
            setProvider(null);
            setBalance(null);
          } else {
            connect();
          }
        };

        const handleChainChanged = () => window.location.reload();

        window.ethereum.on("accountsChanged", handleAccountsChanged);
        window.ethereum.on("chainChanged", handleChainChanged);

        return () => {
          window.ethereum?.removeListener(
            "accountsChanged",
            handleAccountsChanged
          );
          window.ethereum?.removeListener("chainChanged", handleChainChanged);
        };
      }, [connect]);

      return {
        provider,
        signer,
        address,
        chainId,
        balance,
        error,
        connecting,
        isSupported,
        connect,
        refreshBalance
      };
}
```

Ova datoteka služi nam za povezivanje MetaMask računa korisnika sa našom aplikacijom. Prije daljnjeg definiranja funkcije proširujemo želimo osigurati da je Window.ethereum dostupan u TypeScriptu jer nije podržan po default-u, te govorimo TypeScriptu definiciju što je window.ethereum i koje argumente prima. Ako taj korak preskočimo TypeScript bi nam uvijek davao error kada bi pristupili window.ethereum, te kao dodatna funkcionalnost, ako MetaMask nije instaliran u browseru dobit ćemo undefined umjesto errora. Nadalje definiramo WalletState interface odnosno što nam vraća naš useWallet hook. Provider je ethers.js poveznica na čvor za read operacije, Signer je povezani novčanik koji se koristi za write operacije, Address je javna adresa

povezanog novačnika, ChainId identificira mrežu na koju je MetaMask trenutno spojen, Balance je količina ETH-a u trenutnom novčaniku, Error nam služi za čuvanje errora koji mogu doći prilikom spajanja, Connecting nam je flag samo za connect gumb, isSupported provjerava da li imamo adrese ugovora za mrežu na koju je novčanik spojen, connect i refreshBalance su funkcije koje naš hook izlaže za pozivanje. Nadalje definiramo funkcije za povezivanje novčanika i dohvat informacija koliko ETH-a je dostupno u novčaniku. Na kraju koristeći useEffect definiramo dva eventa, accountsChanged i chainChanged. Za accountsChanged, ako korisnik promjeni račun u svom novčaniku, jednostavno povežemo novi račun sa aplikacijom, a za chainChanged, reloadamo aplikaciju kako bi dobili nove informaciju u našem state-u računa, jer sve ovise o tome na koju mrežu je korisnik povezan.

4.2.2. useContracts.ts

```
import { useState, useCallback, useMemo } from "react";
import { ethers } from "ethers";
import {
  ADDRESSES,
  ITEM_REGISTRY_ABI,
  DUNGEON_BATTLE_ABI,
  type ItemData,
  type BattleRecord
} from "../utils/contracts";

export function useContracts(
  signer: ethers.JsonRpcSigner | null,
  chainId: number | null
) {
  const [loading, setLoading] = useState(false);
  const [txHash, setTxHash] = useState<string | null>(null);
  const [txError, setTxError] = useState<string | null>(null);

  const addresses = chainId ? ADDRESSES[chainId] : undefined;

  const itemRegistry = useMemo(() => {
    if (!signer || !addresses?.ItemRegistry) return null;
    return new ethers.Contract(
      addresses.ItemRegistry,
      ITEM_REGISTRY_ABI,
      signer
    );
  }, [signer, addresses]);
```

```
const dungeonBattle = useMemo(() => {
  if (!signer || !addresses?.DungeonBattle) return null;
  return new ethers.Contract(
    addresses.DungeonBattle,
    DUNGEON_BATTLE_ABI,
    signer
  );
}, [signer, addresses]);

const sendTx = useCallback(
  async <T>(
    contract: ethers.Contract,
    method: string,
    args: unknown[],
    overrides: Record<string, unknown> = {}
  ): Promise<T> => {
    setLoading(true);
    setTxHash(null);
    setTxError(null);
    try {
      const estimated = await contract[method].estimateGas(
        ...args,
        overrides
      );
      const gasLimit = (estimated * 120n) / 100n; // +20%

      const tx: ethers.ContractTransactionResponse = await contract[
        method
      ](...args, {
        ...overrides,
        gasLimit
      });
      setTxHash(tx.hash);
      const receipt = await tx.wait();
      return receipt as T;
    } catch (e) {
      console.error("sendTx error:", e);
      const err = e as {
        reason?: string;
        shortMessage?: string;
        message?: string;
      };
      const msg =
```

```
            err.reason ||
            err.shortMessage ||
            err.message ||
            "Transaction failed";
          setTxError(msg);
          throw e;
        } finally {
          setLoading(false);
        }
      },
      []
  );

  const mintItem = useCallback(
      async (itemType: number) => {
        if (!itemRegistry) throw new Error("Contracts not ready");
        const price: bigint = await itemRegistry.MINT_PRICE();
        return sendTx(itemRegistry, "mintItem", [itemType], {
          value: price
        });
      },
      [itemRegistry, sendTx]
  );

  const repairItem = useCallback(
      async (tokenId: number) => {
        if (!itemRegistry) throw new Error("Contracts not ready");
        return sendTx(itemRegistry, "repairItem", [tokenId], {
          value: ethers.parseEther("0.005")
        });
      },
      [itemRegistry, sendTx]
  );

  const getItem = useCallback(
      async (tokenId: number): Promise<Omit<ItemData, "power"> | null> => {
        if (!itemRegistry) return null;
        const raw = await itemRegistry.getItem(tokenId);
        return {
          tokenId,
          itemType: Number(raw.itemType),
          rarity: Number(raw.rarity),
          attack: Number(raw.attack),
          defense: Number(raw.defense),
```

```
        magic: Number(raw.magic),
        speed: Number(raw.speed),
        durability: Number(raw.durability),
        mintedAt: Number(raw.mintedAt)
      };
    },
    [itemRegistry]
  );

  const getPowerScore = useCallback(
    async (tokenId: number): Promise<number> => {
      if (!itemRegistry) return 0;
      const score: bigint = await itemRegistry.getPowerScore(tokenId);
      return Number(score);
    },
    [itemRegistry]
  );

  const getPlayerItems = useCallback(
    async (address: string): Promise<ItemData[]> => {
      if (!itemRegistry) return [];

      const filter = itemRegistry.filters.ItemMinted(address);
      const events = await itemRegistry.queryFilter(filter, 0, "latest");
      const tokenIds = events.map((e) =>
        Number((e as ethers.EventLog).args.tokenId)
      );

      if (tokenIds.length === 0) return [];

      const owners = await Promise.all(
        tokenIds.map((id) => itemRegistry.ownerOf(id).catch(() => null))
      );

      const ownedIds = tokenIds.filter(
        (_, i) => owners[i]?.toLowerCase() === address.toLowerCase()
      );

      if (ownedIds.length === 0) return [];

      const [rawItems, powers] = await Promise.all([
        Promise.all(ownedIds.map((id) => itemRegistry.getItem(id))),
        Promise.all(
          ownedIds.map((id) => itemRegistry.getPowerScore(id))
```

```
        )
      ]);

      return ownedIds.map((tokenId, i) => ({
        tokenId,
        itemType: Number(rawItems[i].itemType),
        rarity: Number(rawItems[i].rarity),
        attack: Number(rawItems[i].attack),
        defense: Number(rawItems[i].defense),
        magic: Number(rawItems[i].magic),
        speed: Number(rawItems[i].speed),
        durability: Number(rawItems[i].durability),
        mintedAt: Number(rawItems[i].mintedAt),
        power: Number(powers[i])
      }));
    },
    [itemRegistry]
  );

  const enterDungeon = useCallback(
    async (tokenId: number, difficulty: number) => {
      if (!dungeonBattle) throw new Error("Contracts not ready");
      const fee: bigint = await dungeonBattle.ENTRY_FEE();
      return sendTx(
        dungeonBattle,
        "enterDungeon",
        [tokenId, difficulty],
        { value: fee }
      );
    },
    [dungeonBattle, sendTx]
  );

  const claimRewards = useCallback(async () => {
    if (!dungeonBattle) throw new Error("Contracts not ready");
    return sendTx(dungeonBattle, "claimRewards", []);
  }, [dungeonBattle, sendTx]);

  const getPlayerStats = useCallback(
    async (address: string): Promise<{ wins: number; losses: number }> => {
      if (!dungeonBattle) return { wins: 0, losses: 0 };
      const [wins, losses] = await dungeonBattle.getPlayerStats(address);
      return { wins: Number(wins), losses: Number(losses) };
    },
```

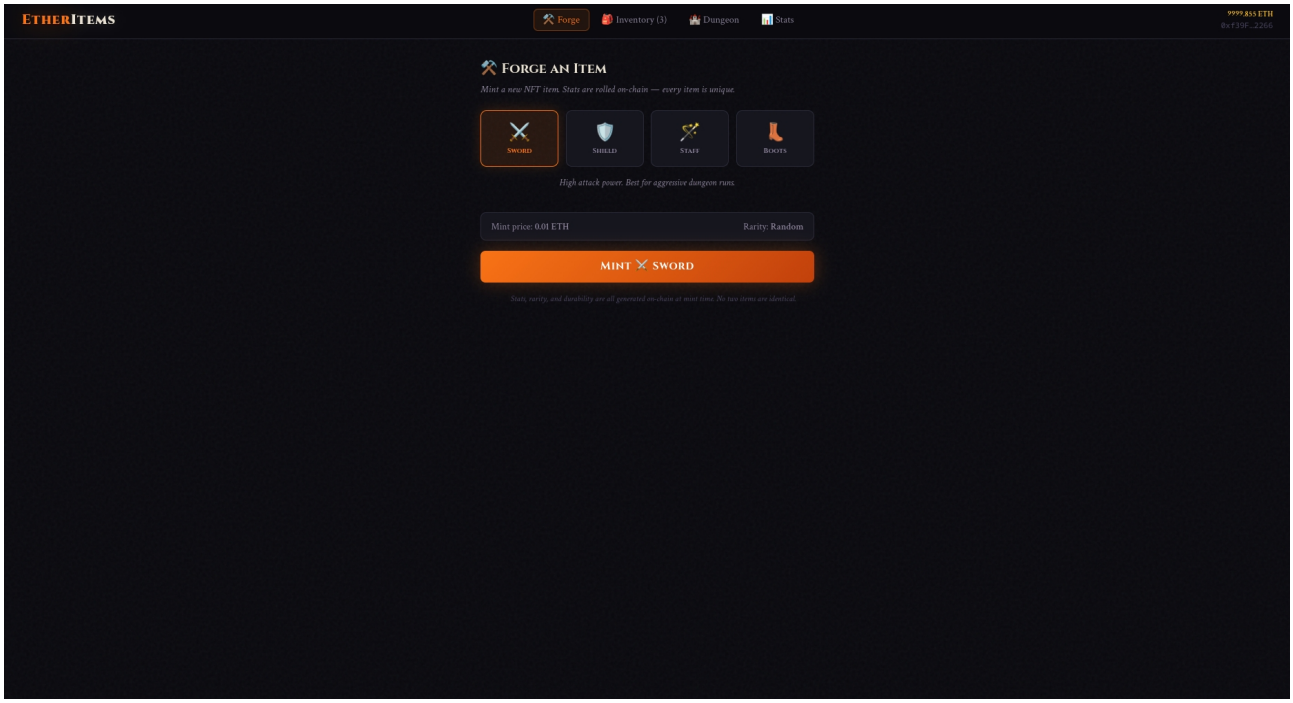
```
    [dungeonBattle]
  );

  const getPendingRewards = useCallback(
    async (address: string): Promise<string> => {
      if (!dungeonBattle) return "0";
      const rewards: bigint = await dungeonBattle.pendingRewards(address);
      return ethers.formatEther(rewards);
    },
    [dungeonBattle]
  );

  const getRecentBattles = useCallback(
    async (count = 5): Promise<BattleRecord[]> => {
      if (!dungeonBattle) return [];
      const raw = await dungeonBattle.getRecentBattles(count);
      return raw.map(
        (b: {
          player: string;
          itemTokenId: bigint;
          playerPower: bigint;
          monsterPower: bigint;
          playerWon: boolean;
          timestamp: bigint;
          monsterName: string;
        }) => ({
          player: b.player,
          itemTokenId: Number(b.itemTokenId),
          playerPower: Number(b.playerPower),
          monsterPower: Number(b.monsterPower),
          playerWon: b.playerWon,
          timestamp: Number(b.timestamp),
          monsterName: b.monsterName
        })
      );
    },
    [dungeonBattle]
  );

  return {
    loading,
    txHash,
    txError,
    setTxError,
```

```
    mintItem,
    repairItem,
    getItem,
    getPowerScore,
    getPlayerItems,
    enterDungeon,
    claimRewards,
    getPlayerStats,
    getPendingRewards,
    getRecentBattles
  };
}
```

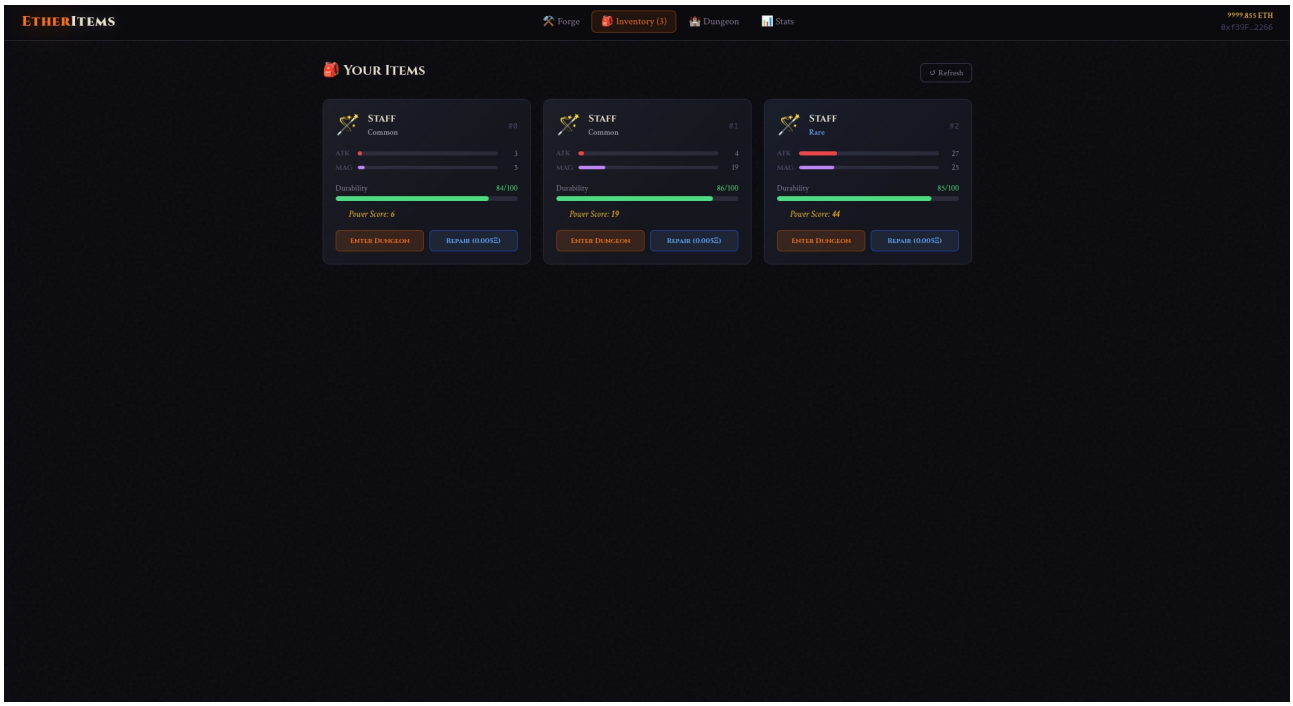
Ova datoteka nam je zapravo glavna datoteka u našem klijentskom dijelu aplikacije, i koja je zadužena za sve funkcionalnosti. Najvažniji dijelovi ove datoteke su instance ugovora, sendTx funkcija, dohvaćanje korisnikovih stvari i konverzija tipova podataka za prikaz na ekranu. Instance ugovora kreiramo pomoću useMemo() što nam zapravo pravi instancu i sprema ih u cache i tamo ostaju dok god se ne promjeni signer iz novčanika ili se ne promjene adrese. Ethers.Contract prima za argumente 3 stvari: adresu ugovora tj. Gdje je ugovor kojeg poziva, ABI tj. Kako da "priča" s tim ugovorom i signer tj. Tko poziva funkcije iz ugovora. SendTx funkcija je glavna funkcija koja odrađuje pozive prema ugovorima i radi tri stvari. Prvo radi pretpostavku koliko gas-a je potrebno za transakciju ako ostanemo bez gas-a u transakciji, ta transakciju će biti vraćena, nakon pretpostavke koliko gasa je potrebno dodamo još 20% na tu pretpostavku jer ethers zna podcijeniti koliko gas-a je zapravo potrebno za transakciju i na taj način osiguramo da transakcija uvijek ima dovoljnu količini gas-a, nakon ta dva koraka tek šalje transakciju i čeka odgovor. Nadalje, funkcija getPlayerItems, zbog načina na koji blockchain radi i s obzirom da nemamo klasičnu bazu podataka da bi znali kome točno pripada koja stvar, moramo raditi malo pretraživanja po blockchainu na način da pretražimo povijest događanja na blockchainu. Naime, svaki put kad se stvar "minta" ItemMinted event se emita na blockchain sa vlasnikovom adresom. QueryFilter skenira sve blokove i vraća samo eventa gdje se vlasnik podudara sa igračevom adresom. Nakon toga provjeravamo da je naš igrač stvarni vlasnik te stvari jer je stvar u međuvremenu mogla promjeniti vlasnike sa ownerOf i na taj način dobivamo sve stvari koje su u vlasništvu našeg igrača. Na kraju, Solidity vraća sve brojeve u obliku JS BigInt-a u ethersu, a naše komponente za prikaz očekuju normalan broj tipa number i zbog toga svu pretvorbu radimo u našem hook-u kako bi komponente za prikaz imale čiste podatke za rad.

5. Slike aplikacije

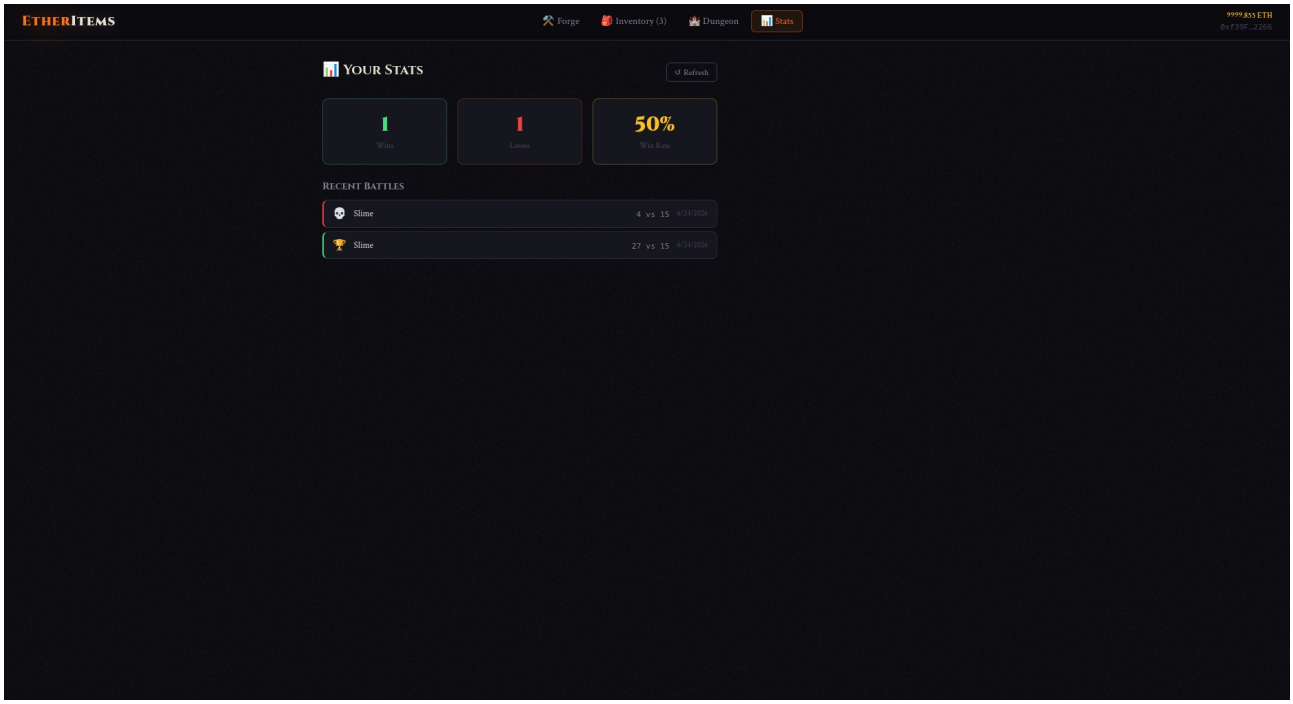
Slika 1. Početna stranica



Slika 2. Forge



Slika 3. Inventory



Slika 4. Battle History i Stats

6. Zaključak

Ova ideja za projekt proizašla je iz znatiželje o web3 igrama i načinu na koji funkcioniraju. S obzirom da je glavna ideja blockchain-a decentralizirani sustav, činilo mi se kao dobra ideja za izradu ovakvog projekta gdje stvari nisu vezane za igrice u kojima se koriste, nego da su jednostavno tvoje i gdje god kreneš nosiš ih sa sobom. Naravno, ovaj projekt je demonstracija skeletona ideje te je moguć i potreban daljnji razvoj kako bi koristio na pravom sustavu, i smatram da bi bilo zanimljivo vidjeti kako bi se takav sustav ponašao u stvarnom svijetu, pogotovo uz mogućnost razvoja neke vrste marketplace-a gdje korisnici mogu prodavati i kupovati stvari, proširiti način popravka stvari pomoću dodatnih alata i slično.

Literatura

Solidity – https://docs.soliditylang.org/en/v0.8.35/

OpenZeppelin Contracts - https://docs.openzeppelin.com/contracts

ERC-721 Non-Fungible Token Standard - https://eips.ethereum.org/EIPS/eip-721

Smart Contract Security Best Practices - https://consensysdiligence.github.io/smart-contract-best-practices/

Hardhat Documentation - https://hardhat.org/docs/getting-started

ethers.js Documentation - https://docs.ethers.org/v6/

OpenZeppelin ERC721 implementation - https://github.com/OpenZeppelin/openzeppelin-contracts